public Stream distinct();

Stream.filter()

It is a predefined method of Stream interface.

If we want to return stream from another stream by removing all the duplicates then we should use filter() method.

```
package com.rest.test;
import java.util.stream.Stream;
public class StreamFilterDemo {
    public static void main(String[] args) {
        Stream<Integer> numbers = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 3, 4, 5);
        numbers.distinct().forEach(System.out::println);
    }
}
```

public Stream<T> limit(long maxSize);

It is a predefined method of Stream interface to work with sequence of elements.

The limit() method is used to limit the number of elements in a stream by providing maximum size.

It creates a new Stream by taking the data from original Stream.

Elements which are in the range or equal to beyond the range of specified limit will be ignored.

```
package com.rest.test;
```

It is a predefined method of Stream interface which is used to skip the elements from beginning of the stream.

It returns a new stream that contains the remaining elements after dropping the specified number of elements which is passed as a parameter.

```
package com.rest.test;
import java.util.stream.Stream;
public class ListFilterDemo {
    public static void main(String[] args) {
        Stream<String> s = Stream.of("Virat", "Rohit", "Dhoni", "Sehwag",
"Yuvi", "Jadeja", "Kohli", "Kumar");
        s.limit(3).forEach(System.out::println);
        System.out.print("\n");
        Stream<String> st = Stream.of("Virat", "Rohit", "Dhoni", "Sehwag",
"Yuvi", "Jadeja", "Kohli", "Kumar");
        st.skip(3).forEach(System.out::println);
    }
}
```

public Stream<T> peek(Consumer<T super T> action);

It is a predefined method of Stream interface which is used to perform a side-effect operation on each element of the stream without modifying the original stream.

It is an intermediate operation that allows us to perform an operation on each element of stream without modifying original.

The peek() takes a Consumer as an argument, and this function is applied to each element in the stream. It returns a new stream that contains the same elements as the original stream.

```
package com.rest.peek;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
public class ReadDemo {
    public static void main(String[] args) {

    List<String> name = Arrays.asList("Ravi", "Sunil", "Raj", "Satish");
    List<String> collect = name.stream()
            .filter(name -> name.length() > 3)
            .peek(name -> System.out.print("filtered: " +
name.toUpperCase()+ "\n"))
            .map(String::toUpperCase)
            .peek(name -> System.out.print("Mapped: " + name))
            .toList();
    System.out.print("\n");
    System.out.print("Collected: " + collect);
    System.out.print("\n");
    }
}
```

```
package com.rest.peek;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
public class PeekDemo {
    public static void main(String[] args) {

    List<Integer> numbers = Arrays.asList(10, 15, 20, 25, 30);
    List<Integer> result = numbers.stream()
            .filter(n -> n % 2 == 0)
            .peek(n -> System.out.print("Even number: " + n))
            .map(n -> n * 2)
            .peek(n -> System.out.print("Doubled value: " + n))
            .toList();
    System.out.print("Final Result: " + result);
    }
}
```

public Stream<T> takeWhile(Predicate<T> predicate);

It is a predefined method of Stream interface which accept from pass the which is used to perform a side-effect operation on each element of the stream without modifying original stream.

TW is used to create a new stream that includes elements from the original stream only as long as they satisfy a given predicate.

```
package com.rest.take_and_drop_while;
import java.util.Arrays;
public class TakeWhileDemo {
    public static void main(String[] args) {

    Stream<Integer> numbers = Stream.of(10,11,12,13,14,15);
    numbers.takeWhile(n -> n % 2).forEach(System.out::println);
    System.out.print("\n");

    numbers = Stream.of(11,12,14,5,6,7);
    numbers.takeWhile(n -> n%2==0).forEach(System.out::println);
    System.out.print("\n");

    numbers = Stream.of(1,3,5,6,7,9);
    numbers = Stream.of(1,3,5,6,7,9);
    numbers.takeWhile(n -> n % 2).forEach(System.out::println);
    System.out.print("\n");

    numbers = Stream.of(1,3,5,6,7,9);
    numbers.takeWhile(n -> n % 2).forEach(System.out::println);
    System.out.print("\n");

    Stream<String> stream = Stream.of("Ravi", "Sunil", "Kumar", "Ramesh");
    stream.takeWhile(n -> n.startWith("R").forEach(System.out::println);
    }
}
```

public Stream<T> dropWhile(Predicate<T> predicate);

It is a predefined method of Stream interface which accept from pass the which is used to create a new stream by dropping elements from the original stream as long as they satisfy a given predicate.

```
package com.rest.take_and_drop_while;
import java.util.stream.Stream;
public class DropWhileDemo {
    public static void main(String[] args) {

    Stream<Integer> numbers = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
    numbers.dropWhile(n -> n < 5).forEach(System.out::println);
    System.out.print("\n");

    numbers = Stream.of(10, 9, 8, 7, 6, 5, 4, 3, 2, 1);
    numbers.dropWhile(n -> n > 5).forEach(System.out::println);
    }
}
```

Terminal Operation in Stream in Java:

In Java's Stream API, a terminal operation is an operation that produces a result because it's a final operation.

Unlike intermediate operations, which return a new stream, terminal operation consumes the elements of the stream.

Once a terminal operation is applied to a stream, the stream is considered consumed and cannot be reused.

Terminal Methods:

1) public long count();

2) public Optional<T> min(Comparator<T> cmp);

3) public Optional<T> max(Comparator<T> cmp);

4) public Optional<T> findFirst();

5) public Optional<T> findAny(); (For better output, use ParallelStream())

6) public boolean anyMatch(Predicate<T> p);

7) public boolean allMatch(Predicate<T> p);

8) public boolean noneMatch(Predicate<T> p);

9) public void forEach(Consumer<T> c);

10) public A[] toArray(IntFunction<A[]> gen); (If methods)

11) public Optional<T> reduce(BinaryOperator<T> f); (Accumulator) (Sum + sum +)

public long count();

The count operation returns the number of elements available in the stream.

It is a terminal operation that terminates the stream after its execution.

Basically it used to count the number of elements after filter() method.

```
package com.rest.count;
import java.util.stream.Stream;
public class CountDemo {
    public static void main(String[] args) {

        long count = Stream.of("Ravi","Raj","Vijay","Aryan","Raj").count();
        System.out.print(count);
    }
}
```

```
package com.rest.count;
import java.util.List;
```

Count the names which length is > 3

```
public class CountDemo {
    public static void main(String[] args) {

    List<String> list = List.of("Ravi","Raj","Vijay","Aryan","Raj");
    long count = list.stream().filter(s -> s.length() > 3).count();
    System.out.print("Names whose length is > 3 are " +count);
    }
}
```

```
package com.rest.count;
import java.util.List;
```

Count the number of unique elements

```
public class CountDemo {
    public static void main(String[] args) {

    List<String> list = List.of("Ravi","Raj","Ravi","Virat","Raj");
    long count = list.stream()
            .distinct()
            .count();
    System.out.print("Count of unique elements: " + count);
    }
}
```

```
package com.rest.count;
import java.util.Arrays;
import java.util.List;
public class CountDemo {
    public static void main(String[] args) {

    List<String> list = Arrays.asList("Ravi","Raj","Ravi","Virat","Raj");
    long count = list.stream()
            .filter(s -> s.contains("a"))
            .count();
    System.out.print("All the names contains character a is " +count);
    }
}
```

public Optional<T> min(Comparator<T super T> comparator);

It is a predefined method of Stream interface. It is used to find the minimum element of the stream according to the provided Comparator.

If the stream is empty, it will return an empty Optional.

```
package com.rest.min;
import java.util.Arrays;
import java.util.List;
import java.util.Optional;
import java.util.Optional;
public class MinDemo {
    public static void main(String[] args) {

    List<Integer> listOfNumbers = Arrays.asList(10, 20, 5, 15, 30, 25, 1);
    Optional<Integer> min = listOfNumbers.stream().min(Integer::compareTo);
    System.out.print("Minimum element is: ");
    min.ifPresent(System.out::print);
    System.out.print("\n");
    int[] ints = {3,2,5,9};
    int[] minInts = ints.clone();
    Arrays.sort(minInts);
    System.out.print("Minimum element is: " + minInts[0]);
    }
}
```

```
package com.rest.min;
import java.util.Comparator;
import java.util.List;
```

Finding the Cheapest Flight

```
record FlightOption(String id, String name, Double p) {
}

public class FlightDemo {
    public static void main(String[] args) {

    List<FlightOption> options = List.of(
            new FlightOption("AI-101", "Air India AI", 187.0),
            new FlightOption("6E-202", "IndiGo 6E", 142.5),
            new FlightOption("SG-303", "SpiceJet SG", 159.0),
            new FlightOption("UK-404", "Vistara UK", 175.0);

    System.out.print("Cheapest flight between AMD to BLR");
    FlightOption cheapest = options.stream()
            .min(Comparator.comparingDouble(FlightOption::p));
            .ifPresent(System.out.print);
    }
}
```

public Optional<T> max(Comparator<T super T> comparator);

It is a predefined method of Stream interface. It is used to find the maximum element of the stream according to the provided Comparator.

This method is useful when we need to find out the largest element in a stream, based on a specific comparator criteria using Comparator.

```
package com.rest.max;
import java.util.Arrays;
import java.util.List;
import java.util.Optional;
import java.util.Optional;
public class MaxDemo {
    public static void main(String[] args) {

    List<String> listOfNames = Arrays.asList("Ravi", "Sunil", "Raj", "Satish");
    Optional<String> max = listOfNames.stream().max(Comparator.comparingInt(String::length));
    System.out.print("Maximum element is: ");
    max.ifPresent(System.out::print);

    System.out.print("\n");
    int[] ints = {3,2,5,9};
    int[] maxInts = ints.clone();
    Arrays.sort(maxInts);
    System.out.print("Maximum element is: " + maxInts[3]);
    }
}
```

```
package com.rest.max;
import java.util.Comparator;
import java.util.List;
```

Finding the Cheapest Flight

```
record FlightOption(String id, String name, Double p) {
}

public class FlightDemo {
    public static void main(String[] args) {

    List<FlightOption> options = List.of(
            new FlightOption("AI-101", "Air India AI", 187.0),
            new FlightOption("6E-202", "IndiGo 6E", 142.5),
            new FlightOption("SG-303", "SpiceJet SG", 159.0),
            new FlightOption("UK-404", "Vistara UK", 175.0);

    System.out.print("Cheapest flight between AMD to BLR");
    FlightOption cheapest = options.stream()
            .max(Comparator.comparingDouble(FlightOption::p));
            .ifPresent(System.out.print);
    }
}
```

public Optional<T> reduce(Comparator<T super T> comparator);

It is a predefined method of Stream interface. It is used to find the maximum element of the stream according to the provided Comparator.

This method is useful when we need to find out the largest element in a stream, based on a specific comparator criteria using Comparator.

```
package com.rest.max;
import java.util.Arrays;
import java.util.List;
import java.util.Optional;
import java.util.Optional;
public class MaxDemo {
    public static void main(String[] args) {

    List<String> listOfNames = Arrays.asList("Ravi", "Sunil", "Raj", "Satish");
    Optional<String> max = listOfNames.stream().max(Comparator.comparingInt(String::length));
    System.out.print("Maximum element is: ");
    max.ifPresent(System.out::print);

    System.out.print("\n");
    int[] ints = {3,2,5,9};
    int[] maxInts = ints.clone();
    Arrays.sort(maxInts);
    System.out.print("Maximum element is: " + maxInts[3]);
    }
}
```

A collect(Collector<T super T, A, R> collector);

It is a predefined method of Stream interface. It is used to transform the elements of a stream into a different type or collection, such as a List, Set, Map, or Array.

The collect() method takes an argument of type Collector(), which is a functional interface that specifies how to collect the elements of the stream. The Collector() can be built using Collectors() utility methods.

1) Collectors.toList(): Collects the elements of the stream into a List.
2) Collectors.toSet(): Collects the elements of the stream into a Set.
3) Collectors.toMap(): Collects the elements of the stream into a Map.
4) Collectors.joining(delimiter): Concatenates the elements of the stream into a String.
5) Collectors.groupingBy(classifier): Groups the elements of the stream into a Map based on the classifier. The stream is a classifier function.
6) Collectors.partitioningBy(Predicate<T>): Partitions the elements of the stream into two groups based on a predicate.

Collector.toMap(Function<T,K> keyMapper, Function<T,K> valueMapper)

It is a collector used to convert a Stream into a Map.

It takes Function keyMapper and Function valueMapper.

Based on the requirement Map will accept key and value pair.

```
package com.rest.tuMap;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;
record Customer(Integer id, String name, Double bill)
{
}

public class TuMapDemo {
    public static void main(String[] args) {

    List<Customer> listCustomers = List.of(
            new Customer(101, "Ravi", 2500.0),
            new Customer(102, "Sunil", 3200.0),
            new Customer(103, "Kumar", 1800.0),
            new Customer(104, "Ramesh", 4500.0);

    Map<Integer, String> collect = listCustomers.stream()
            .collect(Collectors.toMap(Customer::id, Customer::name));

    System.out.print(collect);
    }
}
```